

:

Name : Moshitha Kolanjiyappan

Reg. No. : 192421205

CSA0603 DAA

CASE STUDY QUESTION ASSIGNMENT

CONTACT SEARCH IN MOBILE APPLICATION USING BINARY SEARCH

1. Description of the Real-World Problem

A mobile phone stores thousands of contacts. When a user types a name in the search bar, the application must display the correct contact instantly. Searching each contact one by one would take more time as the contact list grows. Therefore, an efficient searching technique is required.

2.Problem Statement

Design an efficient contact search mechanism for a mobile application using the Binary Search algorithm. The contacts are stored in alphabetical order. The system should quickly locate a required contact while minimizing the number of comparisons and ensuring a fast response even when the contact list contains thousands of entries.

3.Algorithm Used

Binary Search Algorithm

Binary Search follows the Divide and Conquer technique. Instead of checking every contact, it repeatedly divides the sorted contact list into two halves.

Steps:

1. Set the first and last positions of the sorted contact list.
2. Find the middle contact.
3. Compare the required contact with the middle contact.
4. If both are equal, return the contact.
5. If the required contact comes before the middle contact, search the left half.
6. Otherwise, search the right half.
7. Repeat until the contact is found or no elements remain.

4. Justification for Choosing the Algorithm

Binary Search is selected because it provides very fast searching on sorted data. Mobile users expect search results immediately while typing. Unlike Linear Search, which checks every contact one by one, Binary Search eliminates half of the remaining contacts after each comparison. This greatly improves efficiency for large contact lists. Since contacts in mobile applications are generally maintained in alphabetical order, Binary Search is a practical and optimal solution for this problem.

5. Complexity Analysis

Case	Time Complexity
Best case	$O(1)$
Average case	$O(n \log n)$
Worst case	$O(n \log n)$

6. Manual Execution

Sorted Contact List:

Anand, Arjun, Bharath, Divya, Karthik, Meena, Priya, Rahul

Search Contact: Meena

Step 1:

Middle contact = Divya

Compare Meena with Divya.

Meena comes after Divya, so search the right half.

Remaining contacts:

Karthik, Meena, Priya, Rahul

Step 2:

Middle contact = Meena

Compare Meena with Meena.

Contact Found.

Only **two comparisons** were required instead of checking every contact sequentially.

7. Applications

- Mobile phone contact search
- Telephone directory systems
- Digital address books
- Student record management systems
- Employee database search
- Library management systems
- Hospital patient record lookup
- Banking customer databases

8. Advantages

- Very fast searching for large datasets.
- Requires fewer comparisons than Linear Search.
- Logarithmic time complexity improves efficiency.
- Simple and easy to implement.
- Suitable for real-time mobile applications.
- Uses constant extra memory in iterative implementation.
- Provides quick response for user searches

9. Limitations

- The data must always be sorted before searching.
- Inserting or deleting contacts may require maintaining the sorted order.
- Not suitable for unsorted datasets.
- Less effective for frequently changing data where sorting is expensive..

10. Comparison with Other Algorithms

Feature	Binary Search	Linear Search
Data Requirement	Sorted	Sorted or Unsorted
Searching Method	Divide and Conquer	Sequential Search
Best Case	$O(1)$	$O(1)$
Average Case	$O(\log n)$	$O(n)$
Worst Case	$O(\log n)$	$O(n)$
Suitable for Large Data	Yes	No
Number of Comparisons	May Check Every Element	-

Program

Programiz
C Online Compiler

azertion

Programiz PRO >

main.c

12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49

```
12     if (cmp == 0)
13         return mid;
14     else if (cmp < 0)
15         high = mid - 1;
16     else
17         low = mid + 1;
18     }
19 }
20 return -1;
21 }
22
23 int main() {
24     int n, i, pos;
25     char key[30];
26
27     printf("Enter number of contacts: ");
28     scanf("%d", &n);
29
30     char contacts[n][30];
31
32     printf("Enter contacts in sorted order:\n");
33     for (i = 0; i < n; i++) {
34         scanf("%s", contacts[i]);
35     }
36
37     printf("Enter contact name to search: ");
38     scanf("%s", key);
39
40     pos = binarySearch(contacts, n, key);
41
42     if (pos != -1)
43         printf("Contact found at position %d\n", pos + 1);
44     else
45         printf("Contact not found.\n");
46
47     return 0;
48 }
49
```

Output

Clear

Enter number of contacts: 3
Enter contacts in sorted order:
arvind
mashitha
nikesh
Enter contact name to search: nikesh
Contact found at position 3

=== Code Execution Successful ===

11. Conclusion

Binary Search is significantly more efficient than Linear Search for large, sorted contact lists because it repeatedly divides the search space into halves, reducing the search time from linear to logarithmic. This makes it the preferred algorithm for contact search in mobile applications where users expect instant results and smooth performance.